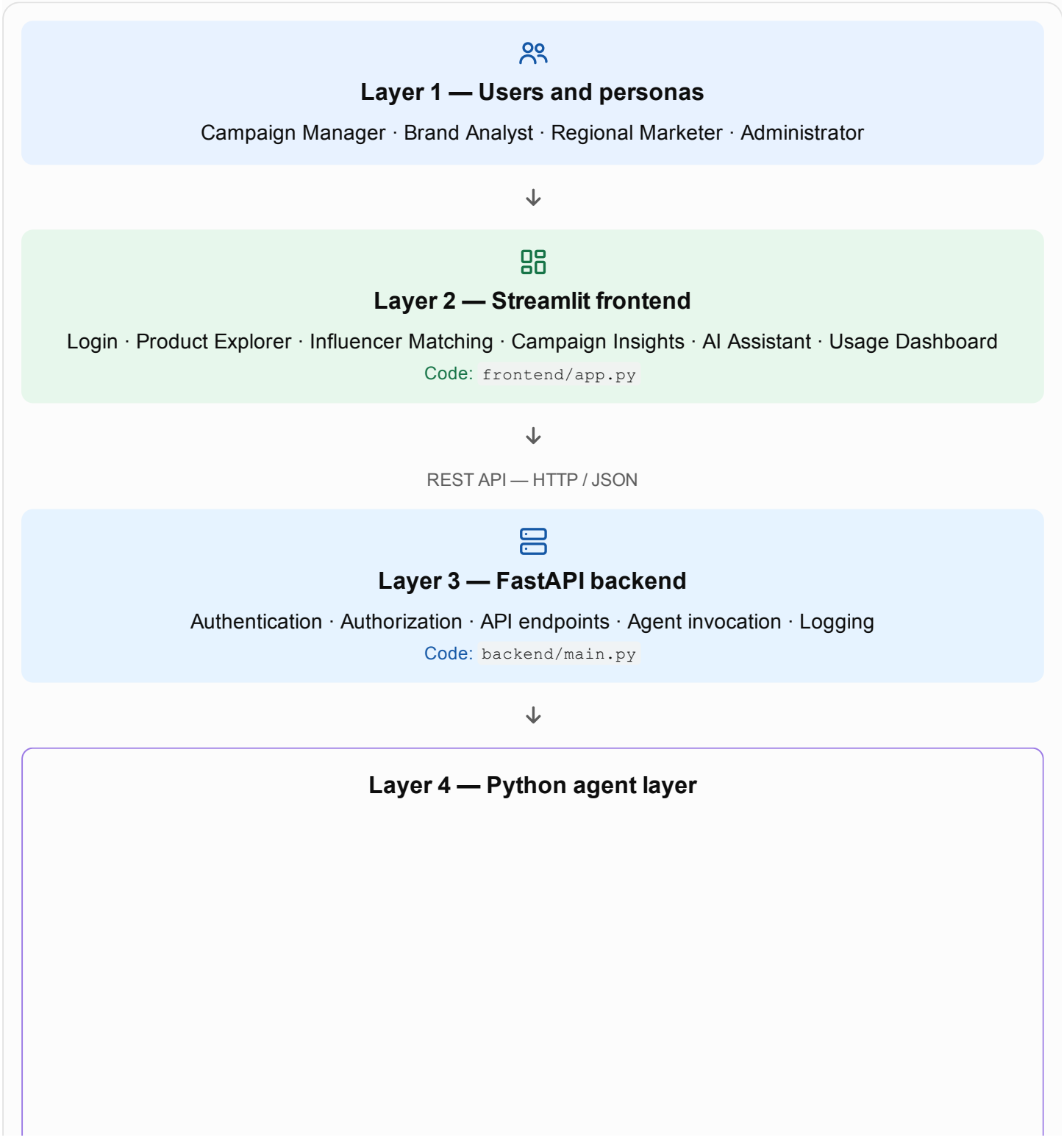
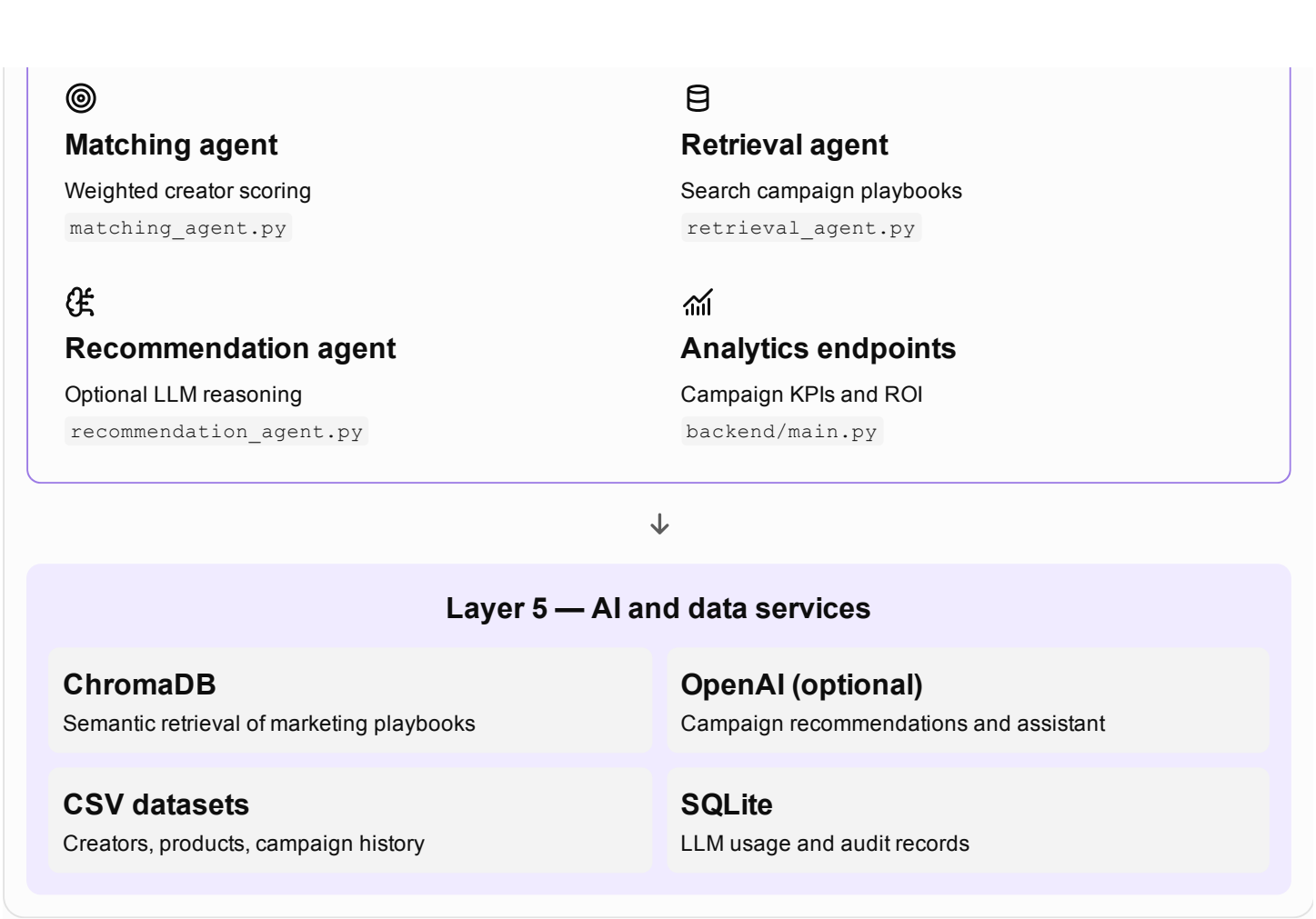
Influencer Matching AI — Architecture explained with Python code

Your solution is a modular Python application that matches products with suitable influencers, ranks creators using audience and campaign data, retrieves marketing guidance from ChromaDB, and uses an LLM when a natural-language recommendation is needed.

The source project uses **Streamlit + FastAPI + Python agents + ChromaDB + optional OpenAI**, with synthetic CSV datasets and persona-based screen restrictions. It does not use MCP or LangChain.

1. Understand the architecture from top to bottom





The arrows represent the normal request flow. The agents are separate Python modules, not separate deployed microservices in this version.

2. Map every architecture component to its source file

Architecture component	Python source	Responsibility
Streamlit UI	`frontend/app.py`	Login screen, navigation, dashboards, matching results
Access control	`shared/security.py`	Demo personas, permitted screens and regional scope
REST API	`backend/main.py`	Receives requests and checks access before calling business logic
Data service	`backend/services/data_service.py`	Reads CSV files and filters data by permitted region
Matching agent	`backend/agents/matching_agent.py`	Calculates influencer-product match scores
Retrieval agent	`backend/agents/retrieval_agent.py`	Searches campaign playbooks using ChromaDB

Recommendation agent	`backend/agents/recommendation_agent.py`	Calls the LLM or returns fallback guidance
Usage and audit service	`backend/services/telemetry.py`	Records tokens, cost estimates, latency and audit events
Synthetic data	`data/*.csv`	Supplies sample creators, products, campaigns and playbooks

3. End-to-end example: how influencer matching works

Suppose a campaign manager wants to promote a beauty product with a budget of $5,000.

1 **The user selects a product in Streamlit**

File: `frontend/app.py`

The UI fetches the available products from the backend and submits the selected `product_id`.

```python
r = api(
    "POST",
    "/match",
    name,
    json={"product_id": pid}
)
```

2 **FastAPI checks access**

File: `backend/main.py`

The `/match` endpoint identifies the user and checks whether their persona has access to the Influencer Matching screen.

```python
u = current(x_username)
need(u, "Influencer Matching")
p, df = run(u, b.product_id)
```

The product and creator data are filtered according to the user's configured region.

3 **The matching agent calculates scores**

File: `backend/agents/matching_agent.py`

It evaluates each creator against the selected product using:

- Audience interest alignment
- Geographic alignment
- Audience age suitability
- Engagement rate
- Brand affinity
- Historical campaign ROI
- Micro-influencer preference

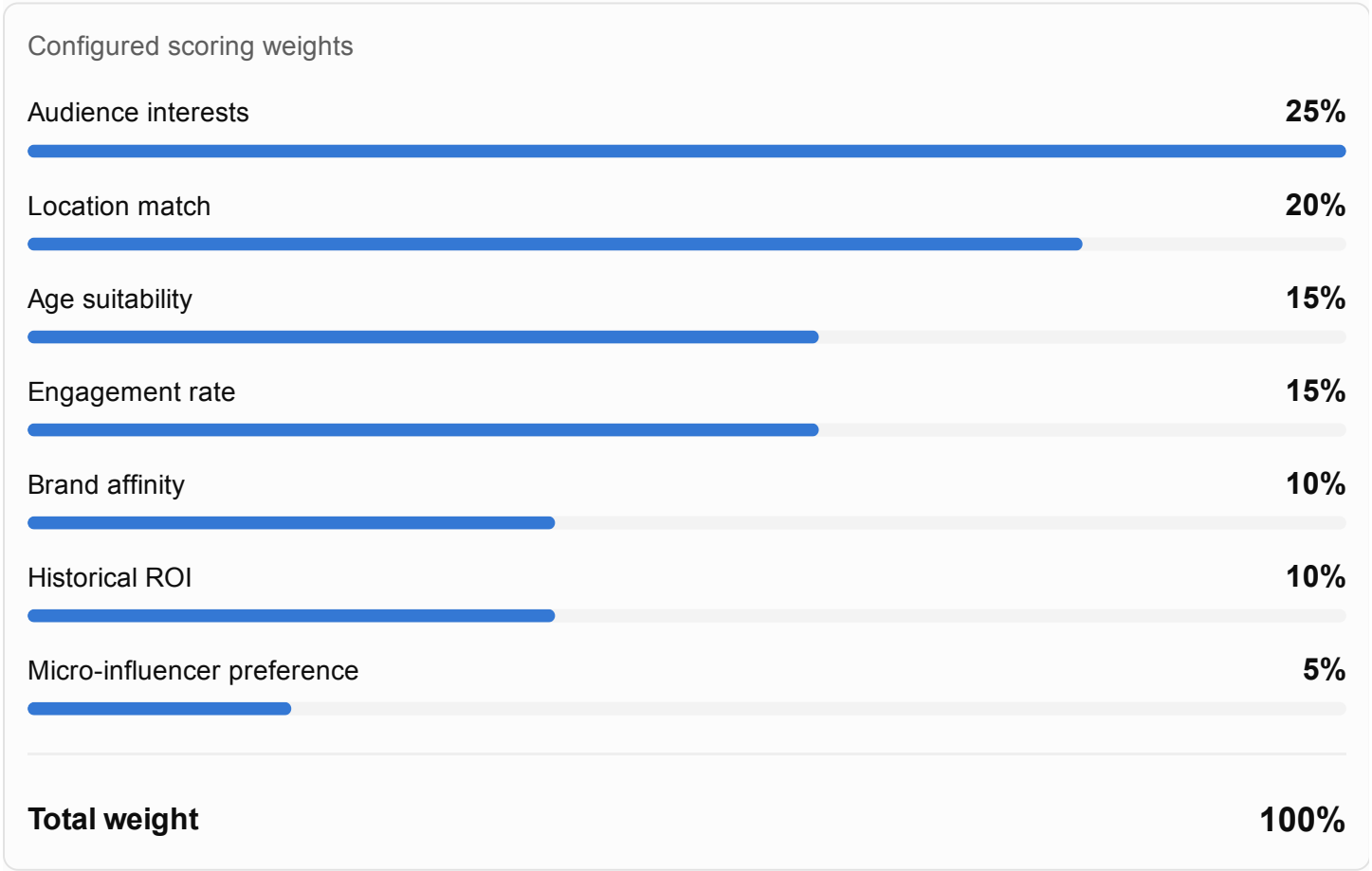
The agent ranks creators, prioritizing those within the campaign budget.

4 **The UI displays ranked creators**

The backend returns JSON containing the product details and creator records. Streamlit displays the results in a table, including match scores, estimated rates and budget eligibility.

How the matching score is calculated

The current implementation uses a weighted rule-based scoring algorithm, not a trained ML model.



Each criterion is normalized to a score, and the weighted scores are combined into a score out of 100. This is a shortlist score, not a probability that a campaign will succeed.

4. How ChromaDB and the LLM work together

These are two different components with different responsibilities.

Step A — Retrieve relevant guidance

File: `backend/agents/retrieval_agent.py`

ChromaDB stores and searches the five synthetic influencer marketing playbooks, such as audience fit, brand safety, budget allocation and ROI measurement.

↓

Step B — Generate a recommendation

File: `backend/agents/recommendation_agent.py`

The recommendation agent sends the product details, ranked creators and retrieved guidance to OpenAI when configured. The LLM explains the trade-offs and proposes a campaign test plan.

↓

Step C — Return the recommendation

FastAPI returns the recommendation and evidence to Streamlit. If OpenAI is not configured, the current code returns predefined fallback guidance instead.

Important distinction: **ChromaDB is the retrieval store; the LLM generates language.** ChromaDB does not itself predict campaign ROI, and the LLM should not be treated as a validated forecasting model.

5. Persona-based access: how screen restrictions work

File: `shared/security.py`

Persona	Permitted screens
Campaign Manager	Dashboard, products, matching, campaign insights, AI assistant, LLM usage
Brand Analyst	Dashboard, products, matching, campaign insights, AI assistant
Regional Marketer	Dashboard, products, matching and campaign insights; regional scope limited to India and Singapore
Administrator	All screens, LLM usage and access audit

The Streamlit application uses the persona to display the appropriate navigation options. The FastAPI backend also checks the requested screen, so hiding a screen in the frontend is not the only access check.

However, the current code uses demo usernames and static passwords, with a username header for subsequent requests. This is suitable for a local prototype, **not production-grade authentication**. Replace it with verified Entra ID/OIDC tokens and validate authorization from those tokens before enterprise deployment.

6. How the LLM Usage Dashboard works

Files: `backend/services/telemetry.py`, `backend/main.py` and `frontend/app.py`.

When the recommendation agent or AI assistant calls the LLM, the service records usage in a local SQLite database named `app_logs.db`.

Request metrics
Number of requests, model, status and purpose

Token and cost metrics
Input tokens, output tokens and estimated cost

Performance
Request latency and usage over time

The current cost calculation uses illustrative rates in the code; replace them with the actual pricing for your selected model. Fallback responses are also logged, but they consume no LLM tokens.

7. What differs between the architecture diagram and the actual code?

This is important when presenting the solution to an architect or technical review board.

Diagram element	Actual implementation
Analytics Agent	Campaign analytics are implemented directly in `backend/main.py`; there is no separate `analytics_agent.py`
ML Models	Influencer matching currently uses weighted rules; no trained ML matching model is included
Application database	SQLite stores usage and audit logs, but user accounts and application configuration are not stored there
Authentication and SSO	Demo credentials only; enterprise SSO is not implemented
AI agents	Separate Python modules, invoked by the FastAPI backend; they are not independently deployed services

ChromaDB	Persistent local vector store; it may initialize a default embedding model on first use
Monitoring	Application-level usage and audit logging, not a complete enterprise monitoring platform

8. Run and explore the code

Download the project: Influencer Matching AI — Complete Python source ZIP ↗

From the extracted project directory, start two terminals.

Terminal 1 — backend:

```powershell
python -m pip install -r requirements.txt
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000 --reload
```

Terminal 2 — frontend:

```powershell
streamlit run frontend/app.py --server.port 8501
```

Open the Streamlit URL shown in your terminal. To explore the backend endpoints, open `http://127.0.0.1:8000/docs`.

My recommendation is to understand the project in this order: `shared/security.py` → `frontend/app.py` → `backend/main.py` → `backend/agents/matching_agent.py` → `backend/agents/retrieval_agent.py` → `backend/agents/recommendation_agent.py` → `backend/services/telemetry.py`.

That sequence follows a real user request from login through matching, AI recommendation and usage logging, making it easier to explain the architecture and debug each layer.